🎓 Infosys Springboard Internship • Milestone 2

Intelligent Cognitive Alarm Platform

A software-verified wake-up framework introducing the **Cognitive Challenge Engine** and stateful **Wake-Up Verification** to eliminate reflexive alarm dismissal.

FastAPI (Python) | React SPA + Vite | SQLAlchemy 2.0 | PostgreSQL / SQLite

INTERN / DEVELOPER

Neha Goswami

EVALUATION TRACK

Infosys Springboard Internship Program

PROJECT SCOPE

Milestone 2: Challenge Engine & Verification

IMPLEMENTATION STACK

FastAPI + React SPA + SQLite/PostgreSQL

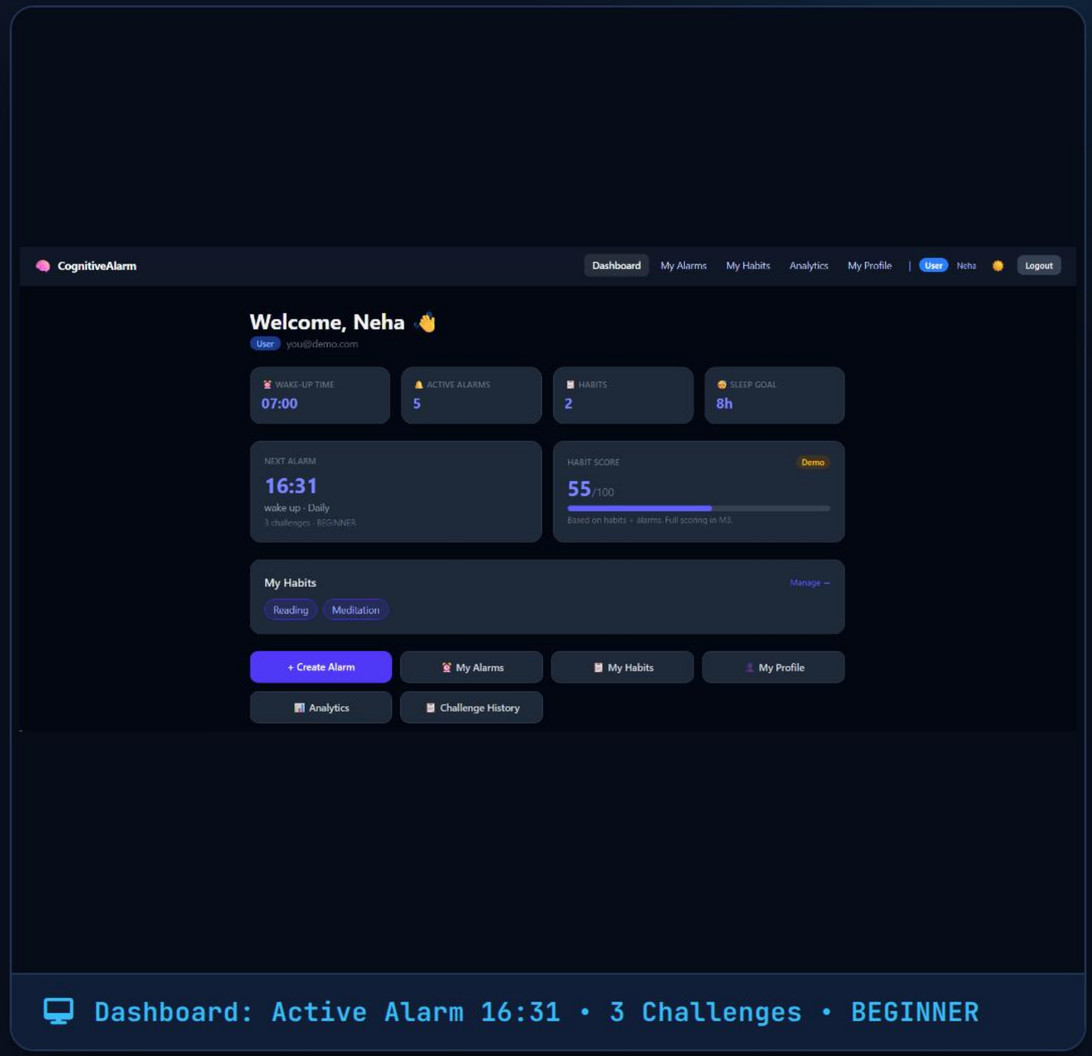
Milestone 2 Objective

M2 • LIVE CONTROL FLOW

Cognitive Engagement vs. Passive Snooze

Standard alarms rely on muscle memory, allowing users to dismiss sound without waking their cognitive faculties.

Milestone 2 inserts an algorithmic puzzle barrier between the trigger and dismissal. Audio can only be silenced once the required problem set is verified as correct by the backend service.



Dashboard: Active Alarm 16:31 • 3 Challenges • BEGINNER

SEQUENTIAL END-TO-END CONTROL FLOW

1. Ring — Alarm fires
2. Challenge — Dynamic gen
3. Attempt — Input sent
4. Evaluate — Strict check
5. Verify — Session lock
6. Silence — Audio off

Problems Addressed in M2

ROOT CAUSE ANALYSIS

Easy Alarm Dismissal

One-tap dismiss allows users to silence alarms in a groggy state without active cognitive engagement.

Repeated Snoozing

Unrestricted snooze loops disrupt sleep hygiene and worsen morning sleep inertia over time.

Zero Mental Interaction

Standard clock apps lack post-wake problem solving, letting users drift back to sleep immediately.

No Telemetry or Metrics

Traditional alarms do not record solve latency, morning cognitive accuracy, or struggle patterns.

Static, Repetitive Tasks

Fixed questions are easily memorized; dynamic generation and difficulty tiers are required.

No Verification Audit

Absence of persistent session records makes it impossible to verify streaks or track wake consistency.

System Architecture

FASTAPI • REACT • SQLALCHEMY

Architectural Request Pipeline

React SPA (Vite + Tailwind) — Client Layer

↓ REST / JSON Over HTTPS

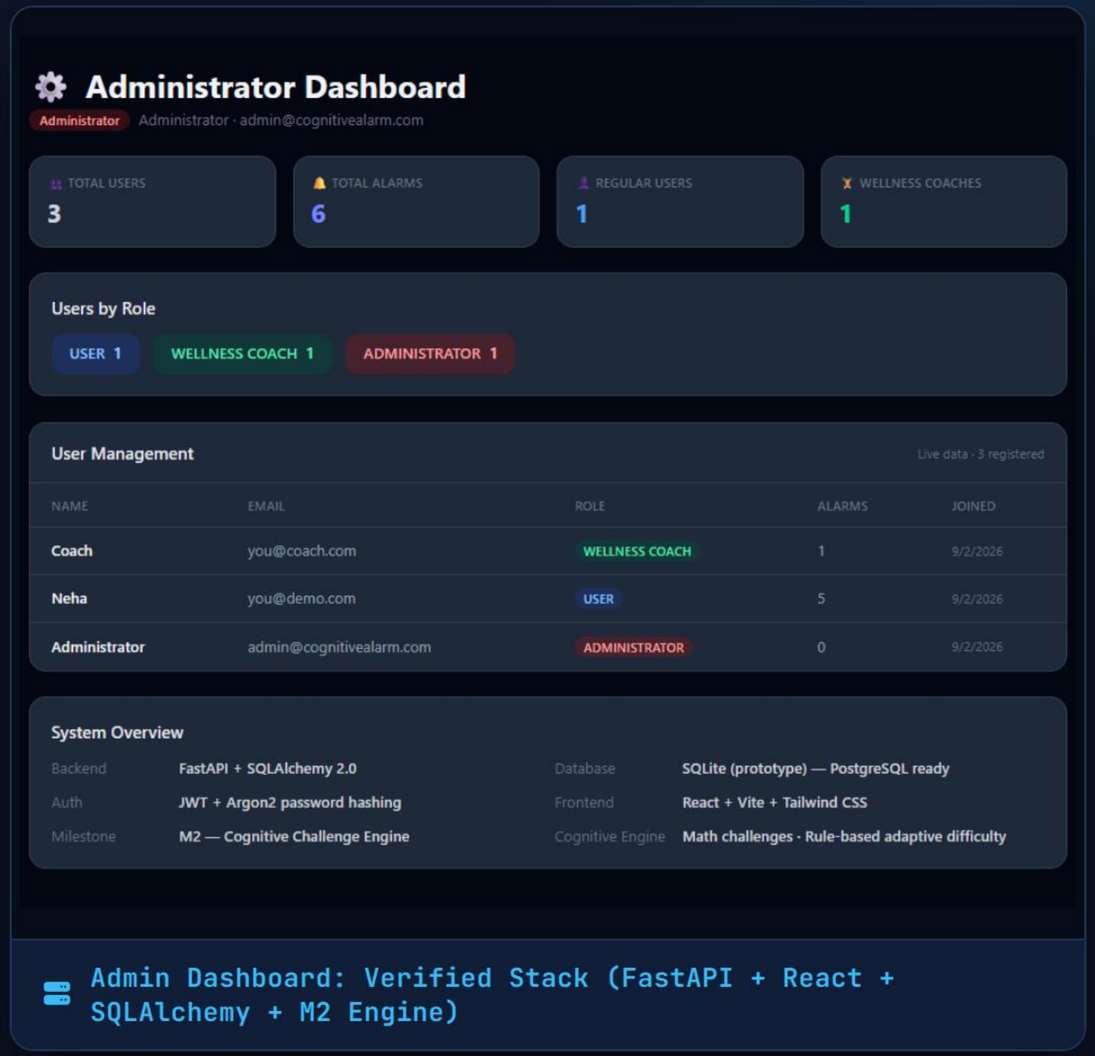
FastAPI Application Server — Async Endpoints

↓ Dispatch & Evaluate

Cognitive Challenge Engine — Puzzles & Verification

↓ Persist Telemetry

SQLAlchemy 2.0 ORM — PostgreSQL / SQLite



Admin Dashboard: Verified Stack (FastAPI + React + SQLAlchemy + M2 Engine)

Challenge Engine Settings

MULTI-CATEGORY FRAMEWORK

Algorithmic Challenge Generation Across 7 Functional Domains:

1. Math Challenges

Arithmetic, operator balancing, multi-step math, and percentages.

2. Logic Reasoning

Deductive premises, truth-table checks, and relational deductions.

3. Memory Sequences

Visual pattern retention and reverse numerical recall.

4. Word & Anagrams

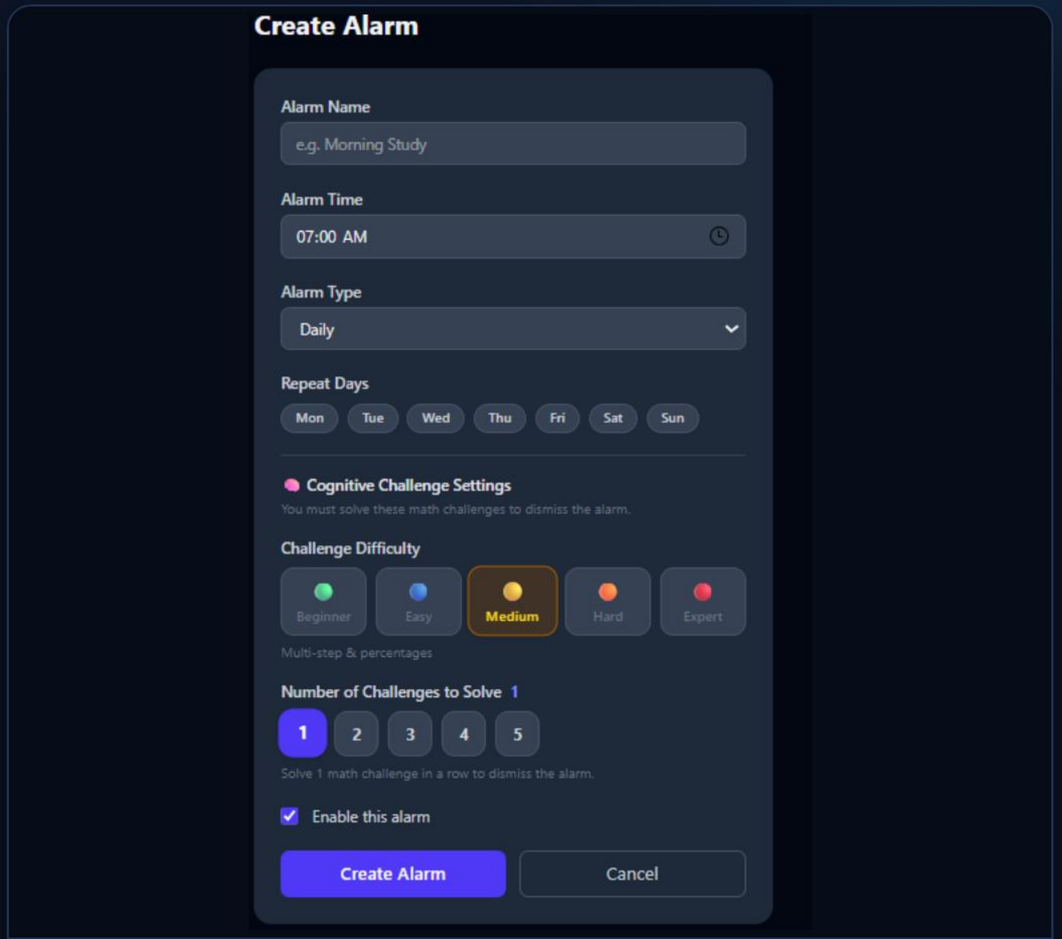
Vocabulary unscrambling and linguistic anagram solving.

5. Pattern & Series

Geometric sequences, odd-one-out matrix, and series progressions.

6. Riddles & Quizzes

Lateral thinking questions and rapid timed multiple-choice.



Alarm Settings: Difficulty Selection & Challenge Count (1 to 5)

5 Implemented Difficulties

COMPLEXITY SCALING

LEVEL 1

Beginner

- **Math:** Single-digit (e.g. 18 − 10 = ?)
- **Memory:** 3-step sequence
- **Logic:** Basic true/false
- **Timer:** 45s generous window

LEVEL 2

Easy

- **Math:** Double-digit (e.g. 8 × 5 = ?)
- **Memory:** 4-step sequence
- **Word:** 4-letter anagrams
- **Timer:** 30s response window

LEVEL 3

Medium

- **Math:** Multi-step (14 × 3 + 12)
- **Memory:** 5-step reverse sequence
- **Logic:** Syllogistic deduction
- **Timer:** 25s response window

LEVEL 4

Hard

- **Math:** Algebraic constraints
- **Memory:** 6-step mixed matrix
- **Pattern:** Geometric progressions
- **Timer:** 20s response window

LEVEL 5

Expert

- **Math:** Modular & complex equations
- **Memory:** 8-step transient grid
- **Logic:** Multi-clause assertions
- **Timer:** Strict < 15s deadline

Rule-Based Adaptive Engine

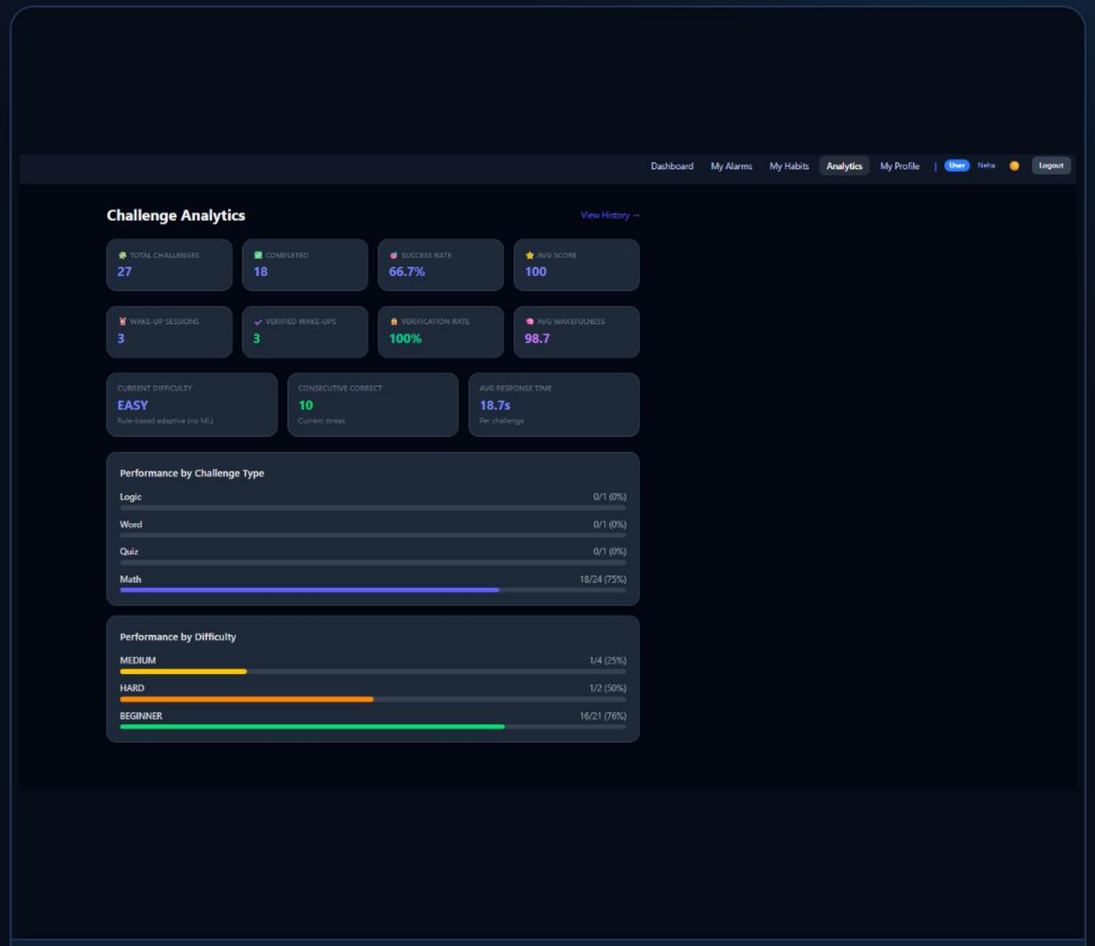
DETERMINISTIC FEEDBACK LOOP

Adaptive Feedback Pipeline

> **Telemetry Inputs:** Accuracy Rate (%) + Response Time (s) + Retries + Streak

- **Promotion Rule (+1 Level):** Consecutive accuracy ≥ 90% and average response latency < 10 seconds.
- **Demotion Rule (−1 Level):** Accuracy ≤ 50% or multiple consecutive challenge timeouts.
- **Steady-State Rule:** Balanced performance retains current tier to ensure optimal waking challenge.

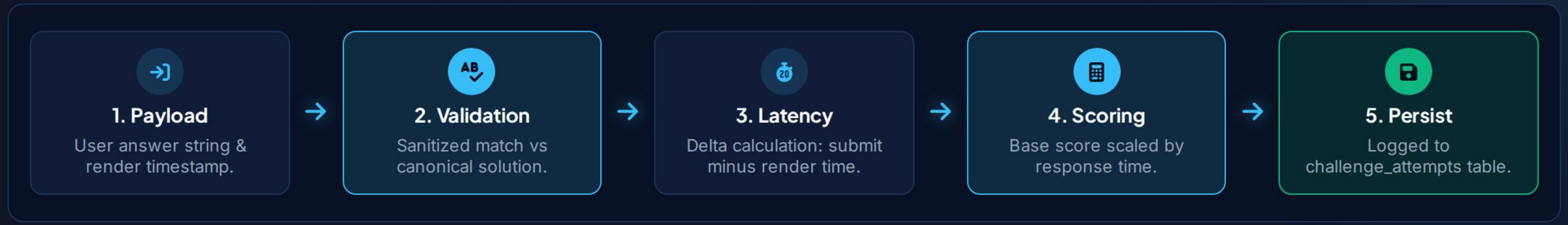
`Deterministic heuristic tuning in Python backend service.`



`Analytics: Verified "EASY • Rule-based adaptive (no ML)" Status`

Challenge Evaluation Flow

CONNECTED EVALUATION PIPELINE



Correctness & Accuracy

Exact match for math computations; sanitized normalization for anagrams, word, and logic solutions.

Dynamic Score Calculation

Score = Base Points × Tier − (Latency × Decay Factor). Bounded to reward swift and precise solving.

Attempt & Retry Tracking

Tracks total attempts per puzzle. Wrong entries increment retry count while keeping the alarm sounding.

Wake-Up Verification Control Flow

STATE MACHINE & LOGIC

1. Scheduled Alarm Fires — AUDIO TRIGGERED

↓

2. Display Challenge & Lock Dismiss — MODAL ACTIVE

↓

3. User Submits Answer & Latency — POST /verify

↓

Decision: Server Validates Solution

INCORRECT

Attempts += 1

Error banner shown.

Alarm audio stays sounding.

Loop back to re-attempt.

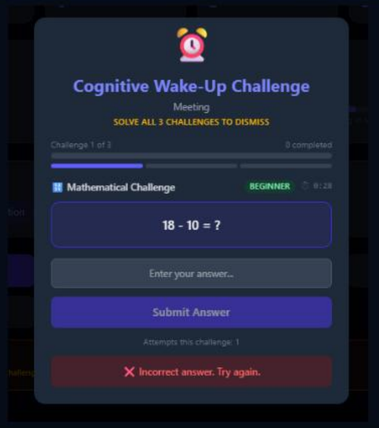
CORRECT

Progress += 1

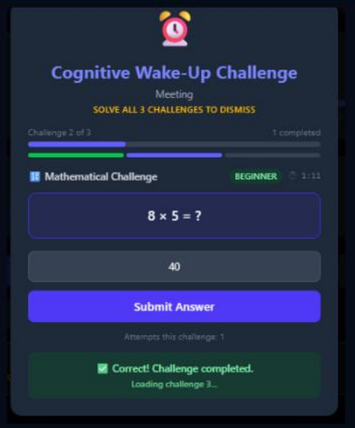
Next challenge loaded until batch requirement (1 to 5) is met.

↓

4. Session VERIFIED: Silence & Dismiss — AUDIO UNLOCKED



Branch A: Incorrect Attempt → "Incorrect answer. Try again."



Branch B: Correct Solution (8 × 5 = 40) → "Loading challenge 3..."

Verification & Wake Score

INTERACTION TELEMETRY

Multi-Step Verification

Requires completing a full configurable batch of 1 to 5 sequential challenges before dismissal.

Time-Window Constraints

Sub-second inputs are flagged as automated guessing; timeouts mark the session challenge failed.

Cognitive Accuracy Ratio

Tracks correct attempts over total attempts (verified at 66.7% across 27 attempts in production logs).

Interaction Wakefulness Score (IWS)

A software interaction score computed from puzzle-solving performance metrics:

```
IWS = (Accuracy × 0.5) + (Normalized_Speed × 0.3) + (First_Try_Rate × 0.2)
```

Software interaction and puzzle response indicator logged per wake-up session.

Database Schema & Models

POSTGRESQL ENTITY MODELS

WAKEUP_SESSIONS

COLUMN	TYPE	DESCRIPTION
id	UUID	Primary Key
user_id	UUID	Foreign Key (users)
alarm_id	UUID	Foreign Key (alarms)
session_status	VARCHAR	PENDING / VERIFIED / FAILED
challenges_req	INT	Required count (e.g. 3)
challenges_done	INT	Completed count
wakefulness_score	FLOAT	Aggregate session score
verified_at	TIMESTAMP	Verification timestamp

COGNITIVE_CHALLENGES

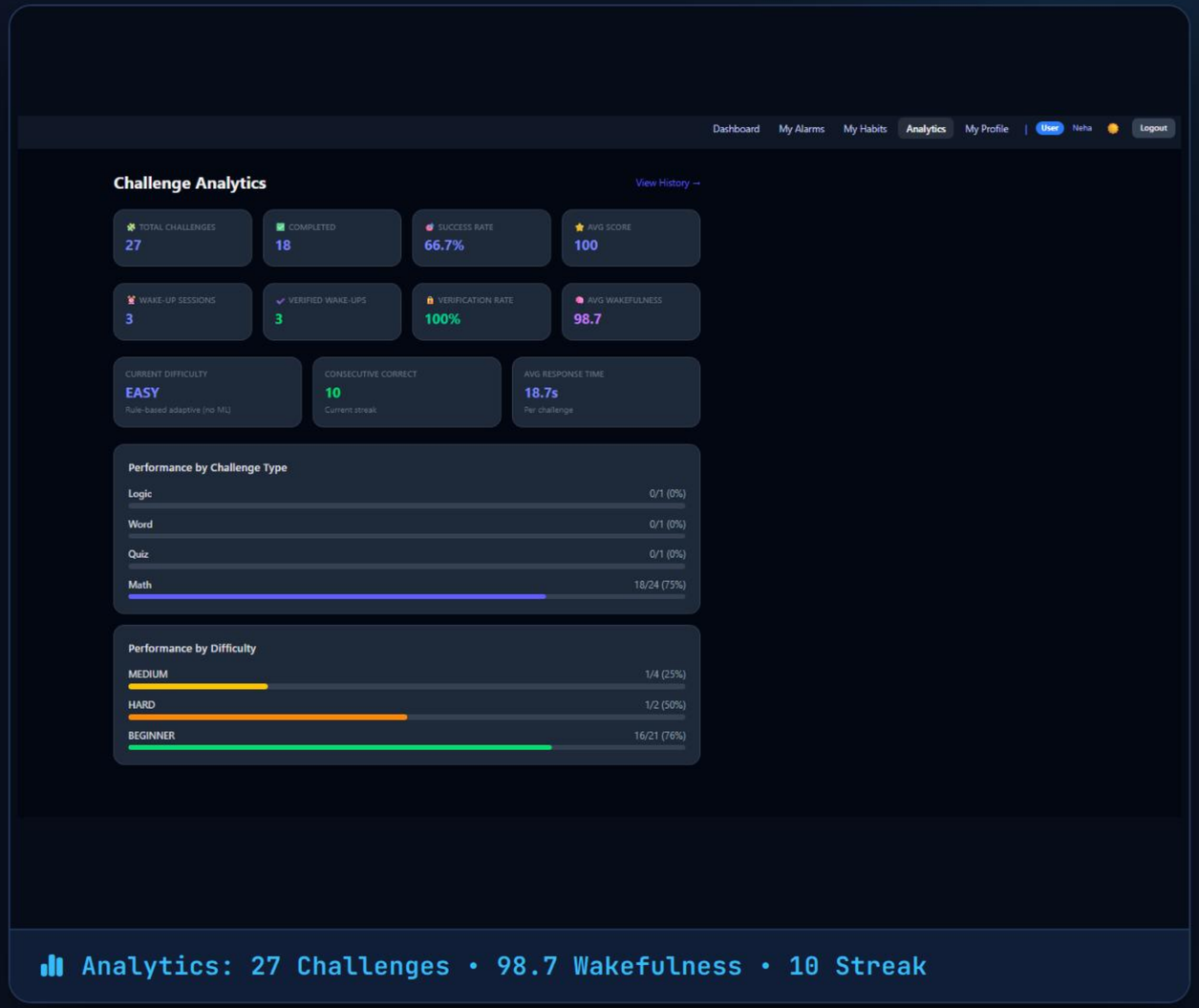
COLUMN	TYPE	DESCRIPTION
id	UUID	Primary Key
category	VARCHAR	Math, Logic, Memory, etc.
difficulty	INT	Level 1 to 5
prompt_payload	JSONB	Question and parameters
canonical_solution	VARCHAR	Expected solution string
time_limit_sec	INT	Timeout limit (seconds)
base_score	INT	Base score value
created_at	TIMESTAMP	Creation timestamp

Analytics & Telemetry

LIVE SYSTEM METRICS

Production Database Telemetry

- **Total Challenges:** 27 attempts with 18 solves (66.7% success rate).
- **Verification Rate:** 100% across 3 completed wake-up sessions.
- **Avg Response Latency:** 18.7 seconds per challenge.
- **Consecutive Streak:** 10 correct answers recorded.



Analytics: 27 Challenges • 98.7 Wakefulness • 10 Streak

Solves by Difficulty Tier

- **Beginner:** 16 / 21 solved (76% success rate)
- **Medium:** 1 / 4 solved (25% success rate)
- **Hard:** 1 / 2 solved (50% success rate)

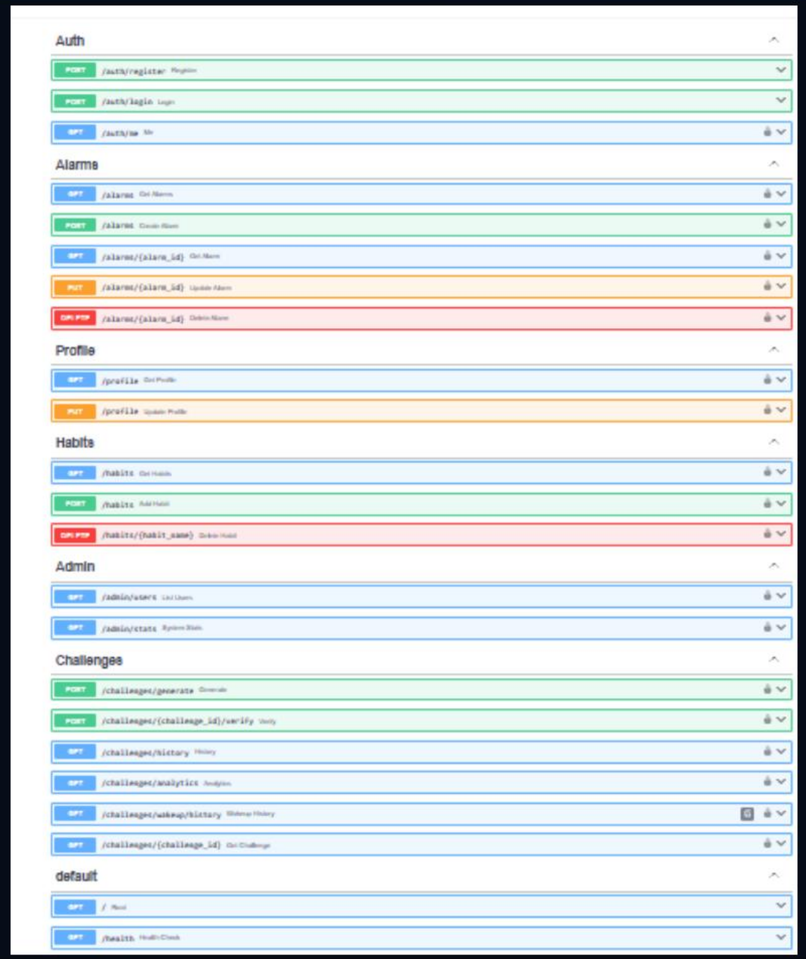
FastAPI Backend Endpoints

REST API ARCHITECTURE

METHOD	ENDPOINT ROUTE	FUNCTIONALITY
POST	/challenges/generate	Generates challenge by category & active difficulty tier.
POST	/challenges/{id}/verify	Validates answer, tracks response latency, and scores attempt.
GET	/challenges/history	Retrieves paginated logs of past challenge attempts.
GET	/challenges/analytics	Computes aggregates: streak, average latency, success rate.
GET	/challenges/wakeup/history	Fetches wake-up sessions and verification outcomes.

Backend Stack Overview

Python 3.11+, FastAPI asynchronous routing, Pydantic data schemas, and Argon2 password hashing.



</> FastAPI Swagger UI: Auth, Alarms, Profile, Challenges

Testing & System Verification

41/41 PYTEST SUITE

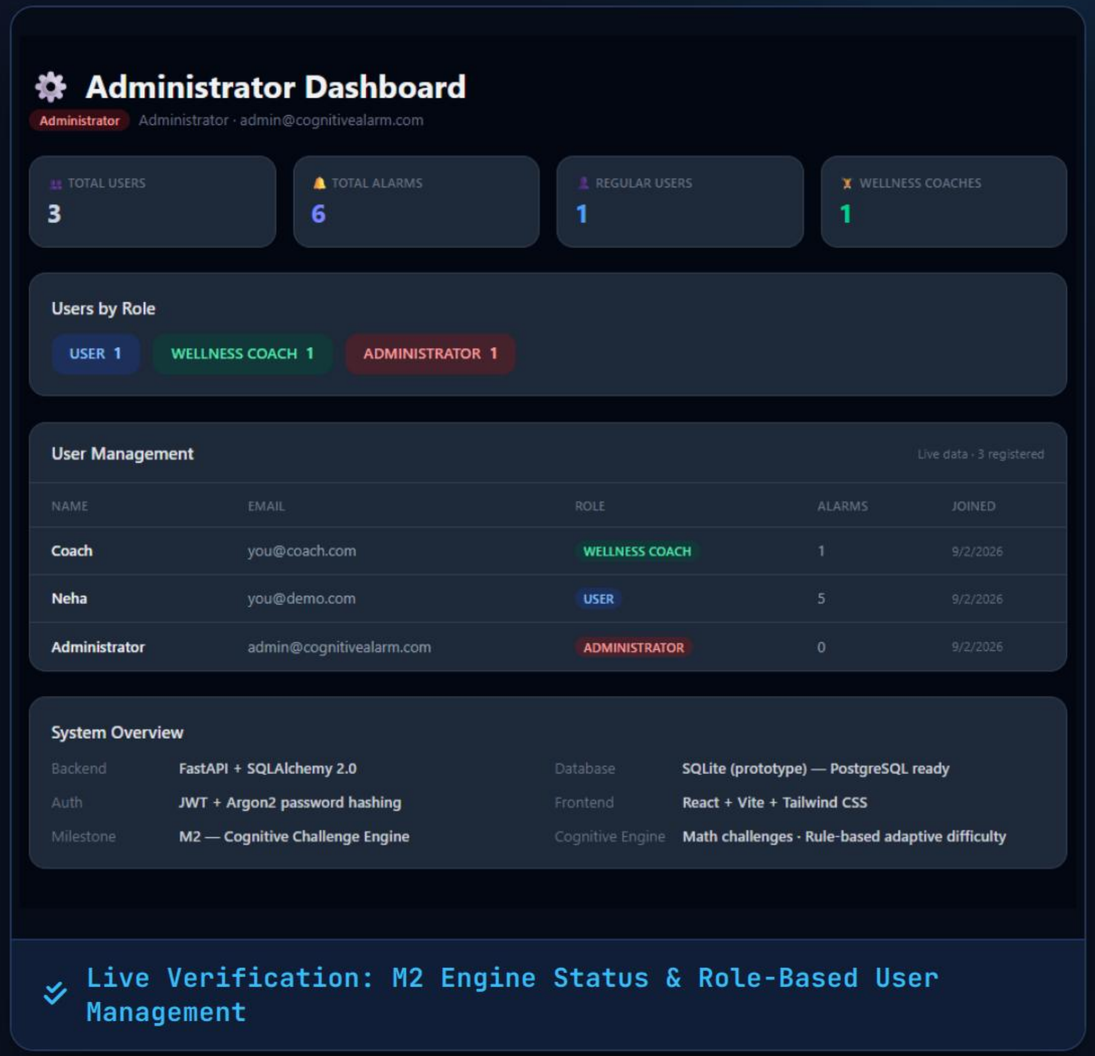
41 / 41

UNIT & INTEGRATION TESTS PASSED (100%)

`pytest v8.x` `All Assertions Green`

Test Coverage Breakdown

- **14 Tests:** Challenge generators across levels and seed variations
- **10 Tests:** Evaluation logic, score decay, and wrong-answer retries
- **9 Tests:** Wake-up session state machine & bypass prevention
- **8 Tests:** History retrieval, pagination, and analytics aggregations



Live Verification: M2 Engine Status & Role-Based User Management

M2 Summary & Future Scope

ROADMAP BOUNDARY

Milestone 2 Delivered (Completed)

- ✓ **7 Challenge Categories:** Math, Logic, Memory, Word, Pattern, Riddle, Quiz
- ✓ **5 Difficulty Tiers:** Beginner, Easy, Medium, Hard, Expert
- ✓ **Evaluation & Retry Logic:** Latency tracking & retry enforcement
- ✓ **Rule-Based Adaptive Engine:** Deterministic difficulty tuning
- ✓ **Wake-Up Verification:** Stateful sessions preventing quick dismiss
- ✓ **Telemetry & Endpoints:** History, streaks & analytics routes
- ✓ **Test Suite:** 41/41 passing unit and integration tests

Future Scope (Milestones 3 & 4)

Explicit Project Boundary: Scope below is planned for subsequent phases.

- **Trained ML Personalization:** Clustering sleep patterns via ML
- **Predictive Inertia Modeling:** Estimating morning grogginess
- **Recommendation Engine:** Optimizing wake windows & difficulty
- **Mobile OS Integrations:** Lockscreen overrides & sensors
- **Production Cloud Deployment:** Distributed microservices rollout